

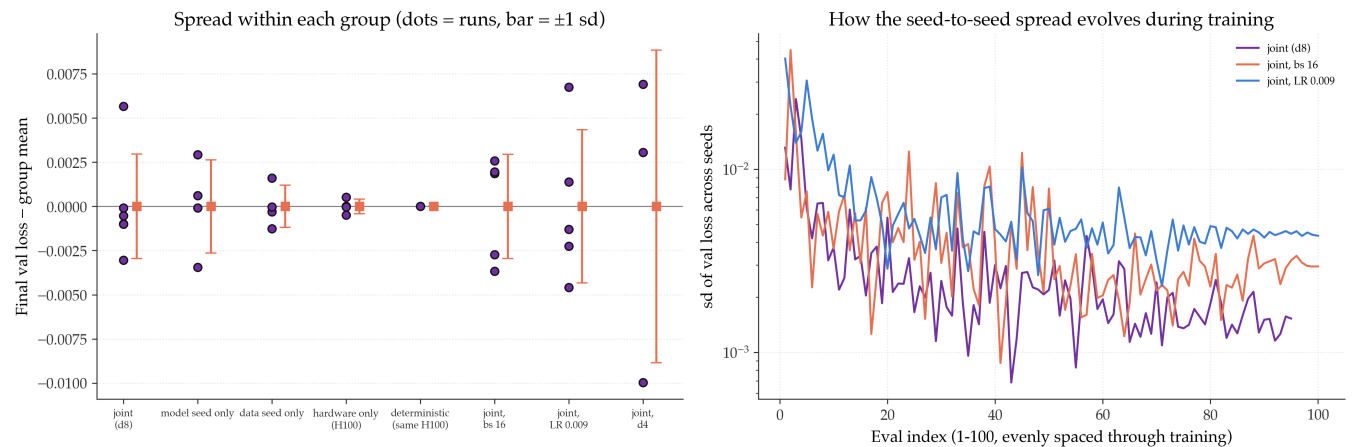
Problems 3-7: Stochasticity, amplification, diagnostics, and our own question

All runs are d8 defaults plus the overrides in the Notes' Experiment code section. Pre-registered predictions: experiments/a1_basics/p3_to_p7_preregistered_predictions.md (recorded before launch).

Problem 3: Measuring variation

Source varied	n	Final val losses	Mean	sd	Range	sd / all-sources sd
All sources (d8, seeds 42,1-5)	6	2.9269, 2.9290, 2.9294, 2.9299, 2.9356, 2.9290	2.9300	0.0030	0.0087	1.00x
Model seed only (42,1,2,3)	4	2.9269, 2.9236, 2.9299, 2.9276	2.9270	0.0026	0.0064	0.89x
Data seed only (42,1,2,3)	4	2.9269, 2.9272, 2.9260, 2.9289	2.9272	0.0012	0.0029	0.40x
Hardware nondeterminism only (H100 x4)	4	2.9269, 2.9264, 2.9264, 2.9259	2.9264	0.0004	0.0010	0.14x
Deterministic, same H100 (x2)	2	2.9277, 2.9277	2.9277	0.0000	0.0000	0.00x
All sources, batch size 16	5	2.9274, 2.9329, 2.9336, 2.9330, 2.9283	2.9310	0.0030	0.0063	1.00x
All sources, LR 0.009	5	2.9528, 2.9581, 2.9501, 2.9468, 2.9491	2.9514	0.0043	0.0113	1.47x
All sources, d4	3	3.2884, 3.2715, 3.2845	3.2815	0.0088	0.0169	3.00x

Cross-hardware (seeds 42/42)	H100	A100	Difference
Nondeterministic (default)	2.9269	pending	pending
deterministic=True	2.9277	pending	pending



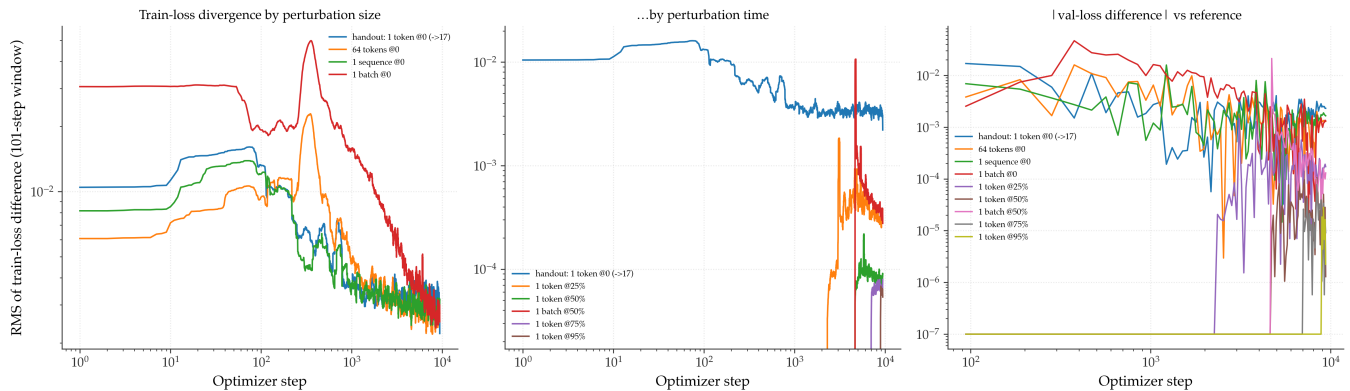
Left: each run's final loss minus its group mean (dots) and ± 1 sd (bars), so spreads are comparable across groups. Right: seed-to-seed sd of val loss at each of the 100 evals through training.

Synthesis. (a) Varying all sources, final loss spreads with sd 0.0030 (range 0.0087); with 6 samples the shape is consistent with a narrow, roughly normal distribution plus an occasional high outlier (seed 4). The loss curves themselves are almost indistinguishable: the spread is ~ 0.01 - 0.02 early and shrinks as the LR decays. (b) **Initialization dominates:** model seed alone gives sd 0.0026 (89% of the joint sd), data order alone 0.0012 (40%), and GPU nondeterminism alone only 0.0004 (14%; two of four repeats were identical). deterministic=True is bit-reproducible on one GPU and lands at a different but fixed value (+0.0008 vs the default, within seed noise; it also disables TF32 and swaps kernels); across GPU types the same seeds give A100 pending (nondeterministic) and pending (deterministic) vs H100. (c) Batch size 16 leaves the spread unchanged (1.0x), LR 0.009 raises it modestly (1.5x), and the smaller d4 is ~ 3 x noisier (sd 0.0088): **more capable models are less variable**. Rule of thumb for d8: differences below ~ 0.006 (2 sd) need seeds.

Problem 4: Amplification of randomness

All runs deterministic=True with default seeds; reference-1 and reference-2 are bit-identical at all 9,375 steps (final val 2.927720 both), so any difference is caused only by the perturbation.

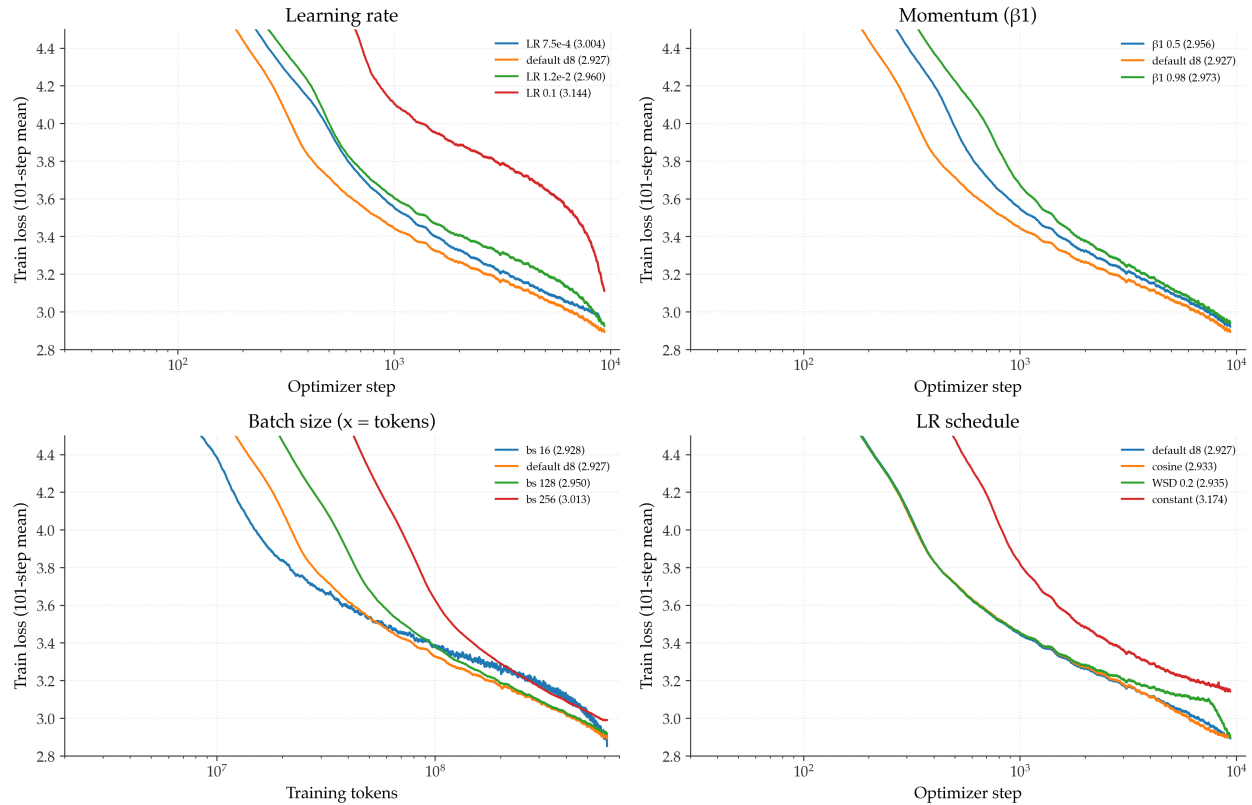
Perturbation	Applied at step	Final val diff vs reference	Peak val diff
handout: 1 token @0 (->17)	0	-0.00231	0.01701
64 tokens @0	0	-0.00130	0.01599
1 sequence @0	0	-0.00166	0.01589
1 batch @0	0	-0.00132	0.04684
1 token @25%	2343	+0.00017	0.00147
1 token @50%	4687	-0.00000	0.00011
1 batch @50%	4687	+0.00013	0.02125
1 token @75%	7031	+0.00001	0.00008
1 token @95%	8906	-0.00000	0.00003



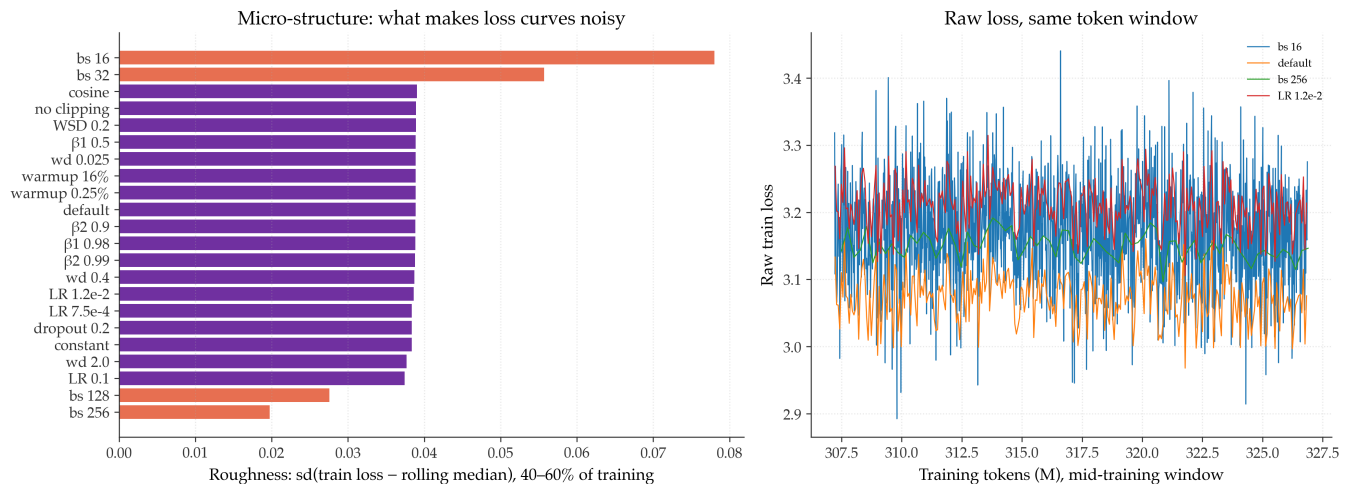
Left/middle: RMS (101-step window) of the per-step train-loss difference from the reference. Right: |val-loss difference| at each eval.

Synthesis. (a) Changing a single token at step 0 changes the final val loss by ~ 0.002 , the same order as seed noise: the perturbation is amplified to a **saturated** level within ~ 10 steps (train-loss difference ~ 0.01) and stays at the noise level for the rest of training. Its size barely matters (1 token, 64 tokens, a sequence, a whole batch all end at 0.0013-0.0023). (b) **Timing matters far more than size:** one token at 25% of training ends at 0.0002, at 50%/75%/95% at < 0.00002 , and even a whole scrambled batch at 50% ends at 0.0001. Amplification is fast early (high LR, rapidly changing loss) and weak late (LR decaying to zero), so late perturbations never reach saturation. Implication: a loss difference of 0.001 between two d8 runs is noise; 0.0001 is noise only if the runs diverged late.

Problem 5: Loss-curve augury



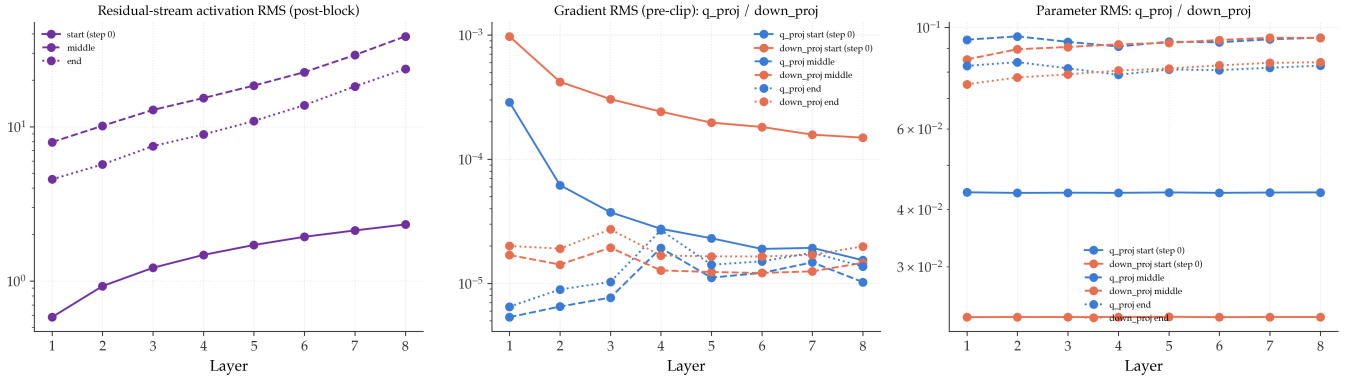
Train loss (101-step mean) for LR, momentum, batch size (x-axis in tokens) and schedule sweeps; final val loss in the legend.



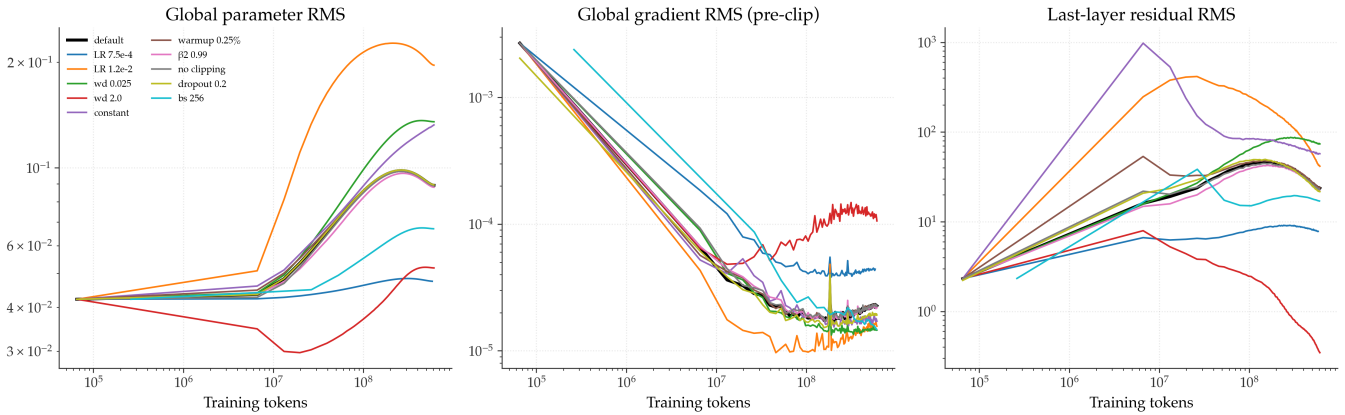
Roughness = sd of train loss around a 51-step rolling median over 40-60% of training.

Synthesis. (a) Macro shapes are predictable: every run shows a fast initial drop to ~ 4.5 , a long power-law-like descent, and - with a decaying schedule - a final 'annealing' drop. LR sets the height of the plateau during training (too high sits above the default until decay rescues it; LR 0.1 barely descends until the end); batch size shifts curves along the token axis (big batches start slower per token); constant LR never gets the final drop; WSD's drop is concentrated in its last 20%. Momentum: β_1 0.5 ends at 2.956 and β_1 0.98 at 2.973 (default 2.927) - both worse, with the high-momentum run slower in the middle. (b) Micro-roughness is set almost entirely by **batch size**: 0.078, 0.056, 0.039, 0.028, 0.020 for bs 16-256, i.e. exactly $\propto 1/\sqrt{\text{bs}}$. Every other knob (LR 7.5e-4 to 0.1, β_1 , β_2 , clipping, wd, warmup, dropout, schedule) stays at 0.037-0.039: step-to-step wiggle is the sampling noise of measuring loss on 64 sequences, not optimizer noise. (c) Runs that changed shapes most: LR 0.1 (flat then late plunge), bs 1024 (long early plateau near loss 6.9), 16x repeated data (train loss keeps falling while val turns up), constant LR (no final drop), tiny-data runs (still steep at the end). Good training = smooth monotone descent with a clean annealing drop; bad = plateaus, rising val, or a missing final drop.

Problem 6: Activations and gradient norms



Default d8 run: per-layer RMS at step 0, mid-training and the end (log scale).



Global parameter RMS, global (pre-clip) gradient RMS and last-layer residual RMS vs tokens for interventions.

Synthesis. (a) In the default run the residual stream grows with depth at every point in training (layer $1 < 3 < 6$), rises from ~ 0.6 - 2 at initialization to ~ 8 - 40 mid-training, then shrinks to ~ 5 - 24 as the LR decays and weight decay acts. Gradient RMS falls $\sim 100\times$ over the first ~ 10 M tokens and then flattens; parameter RMS roughly doubles then relaxes at the end. (b)/(c) Interventions move these statistics predictably: **LR scales everything up** (LR $1.2e-2$: parameter RMS 0.20 vs 0.09 , residual peaks ~ 400), **weight decay scales parameters and activations down and gradients up** (wd 2.0 : residual collapses to 0.35 , gradient RMS $5\times$ higher; wd 0.025 : residual 74). Changes **only at the start**: removing or shortening warmup produces an early activation spike (constant LR, no warmup: residual ~ 1000 at step 100 ; 0.25% warmup: ~ 54 vs 16) that later recovers. Changes **only at the end**: the schedule - linear decay shrinks parameters and activations in the last third, while constant LR keeps them growing. Batch size 256 (fewer steps) keeps parameters smaller throughout.

Problem 7: Our own prediction problem

Question (medium). Every model below sees the same 614.4M training tokens, but only 150,000 unique sequences (153.6M tokens) repeated for 4 epochs. For reference, with **unique** data the final losses are d4 3.290, d6 3.057, d9 2.885 (bigger is better), and with **16x** repetition of 37,500 sequences they are d4 3.393, d6 3.367, d9 3.640 (a U-shape: d9 is worst). Rank d4, d6 and d9 at **4x** repetition from lowest to highest final validation loss, and say whether d9 lands within 0.05 of its unique-data loss.

experiments/a1_basics/p7_own_question.py

```
RUNS = [TrainConfig(model_config=depth_model_config(d), num_train_sequences=150_000,
                    num_epochs=4.0) for d in (9, 6, 4)]
launch_training_jobs(RUNS) # resolves to each run's final val_loss
```

Depth	Unique data	4x repetition (this question)	16x repetition	4x cost vs unique
d4	3.290	3.303	3.393	+0.013
d6	3.057	3.081	3.367	+0.024
d9	2.885	2.933	3.640	+0.048

Answer: d9 < d6 < d4; d9 is +0.048 from its unique-data loss. Pre-registered prediction: d9 < d6 < d4 with d4 ~3.31, d6 ~3.09, d9 ~2.96.

Why it's a good question. Students who memorized '16x repetition makes bigger models worse' will predict a U-shape, and students who memorized 'bigger is better' will ignore the repetition cost. The insight is that repetition cost scales with how many times each token is revisited relative to model capacity: at 4 epochs the larger models still win (memorization has not taken over), but everyone pays a cost that grows with size.

Example quiz questions: answers

Question	Answer	Evidence (run results)	Why
P1 easy: rank A (LR 3e-3, bs 64), B (1e-3, bs 128), C (9e-3, bs 128)	A < C < B (verified)	A 2.927, C 2.992, B 3.025	Doubling batch halves the steps; LR must stay near 3e-3 (1(b)); best LR at bs 128 is 3e-3). Too-low LR (B) is punished more than too-high (C) - the 1(a) asymmetry.
P1 medium: LR 9e-3, wd 1.0; A wsd0.2/warmup 0, B wsd0.2/warmup 0.2, C linear/warmup 0.1	C < B < A (verified)	C 3.060, B 3.206, A 3.364	At high LR warmup is the stability ceiling (1(b)); no warmup (A) is worst. Linear decay beats WSD at high LR (1(c)); B's 20% warmup + 20% decay leaves only 60% of steps at a too-high peak.
P2 hard: d10 gap (scaled - fixed context), pilots predict +0.055	-0.15 < Δ ≤ 0.03 ; don't trust the fit (not run)	Reasoning only (context scaling not supported)	The pilot gap comes from training at shorter context than the 1024-token validation (d4-d6 scaled context < 1024). It vanishes at d8 (identical recipes) and d10 trains on longer context. The d6 scaled curve is also overfitting under 15 epochs (2(c): repetition breaks scaling), so the fitted functional form doesn't apply.
P3 medium: which change more than doubles the seed sd - bs 16, LR 0.009, both, neither?	4. Neither (verified, 5 seeds each)	sd: standard 0.0030, bs 16 0.0030 (1.0x), LR 0.009 0.0043 (1.5x)	Final-loss spread is dominated by initialization and damped by the LR decay to zero; smaller batches add per-step noise but that noise is averaged over 4x more steps. Higher LR adds a little chaos (1.4x), not 2x.
P5 easy: (a) which config made the spiky, fast-dropping curve; (b) its terminal gap	(a) D, LR 0.009; (b) a, at most +0.05	LR 0.009 over 5 seeds: 2.951 (+0.021 vs same-seed standard mean 2.930)	Fast initial drop with visible early spikes is the high-LR signature (bs 256 is smooth and slow per step; LR 3e-4 is slow). LR 0.009 is only 3x the optimum, inside the flat region of the 1(a) LR curve.
P6 easy: residual RMS 1e3-1e5 at layers 1/3/6 - standard d8 or alternative?	Alternative	Default d8 residual RMS: 0.6-1.9 at init, 8-23 mid-training, 5-14 at the end (layers 1/3/6)	Healthy post-block residual RMS is O(1-40); four to five orders of magnitude larger, jumping within the first ~100 steps, indicates broken hyperparameters (the no-warmup constant-LR run peaked at only ~1000).

Every verifiable example matched its pre-registered prediction. Pre-registered predictions that missed elsewhere: hardware-only noise was smaller than predicted (sd 0.0004 vs 0.001-0.002), d4 was ~3x noisier than d8 (predicted within 1.5x), and β1 0.98 cost +0.046 (predicted within +0.02).